

Multi-Tier Sports Betting & Agent Exchange Architecture

Complete Technical Blueprint, Database Schema, Real-Time Order Book, and System Specification

SYSTEM ARCHITECTURE SPECIFICATION

This document details the exact engineering architecture, database layout, credit distribution ledger, risk mitigation rules, and autonomous code execution directives required to build an enterprise-grade multi-tier sports betting exchange.

1. EXECUTIVE PRODUCT REQUIREMENT DESCRIPTION (PRD)

1.1 Target System Overview

The platform is a multi-tenant, high-concurrency sports exchange and credit-based agent management system. Unlike traditional sportsbooks operating with direct fiat gateways, this system leverages a 5-tier hierarchical credit distribution ledger (Admin → Super Master → Master → Agent → User) with real-time exposure calculation, order matching, and automated risk control.

1.2 Functional Hierarchy & Control Matrix

Role Level	Permissions & Administrative Scope	Credit & Exposure Authority
Level 0: Global Admin	Full platform access, system-wide kill switches, event odds stream control, market creation, global commission rules.	Unlimited credit generation, direct limit allocations to Level 1 Super Masters.
Level 1: Super Master	Branch management, creation and suspension of Level 2 Masters, sub-network audit logs.	Allocates credit lines down to Masters. Holds branch-level exposure accountability.
Level 2: Master	Agent oversight, creation/management of Level 3 Agents, sub-channel report tracking.	Allocates credit to Agents within allocated parent limits.
Level 3: Agent	Direct player onboarding, player credential provisioning, password resets, individual user lockouts.	Allocates chips/credits to player accounts. Manual deposit/withdrawal reconciliation.
Level 4: Player / User	Access to exchange frontend, live back/lay wagering, active positions view, ledger statement.	Places bets strictly within assigned credit limit and active exposure bounds.

2. COMPREHENSIVE TECHNICAL & DEPLOYMENT STACK

Backend & High-Frequency Engine

- **Core API:** Node.js (TypeScript) / Express for REST endpoints.
- **Matching Engine:** Go (Golang) microservice for memory-speed Back/Lay order matching.

Frontend Ecosystem

- **Player Web App:** Next.js 14+ (React, Tailwind CSS, Zustand state management).
- **Agent / Admin Portal:** React + Vite SPA, Shadcn UI, TanStack Table for high-density financial data.

- **Real-time Layer:** Socket.io / Native WebSockets with Redis Pub/Sub for sub-50ms odds distribution.
- **Task Workers:** BullMQ with Redis for asynchronous settlement and notification processing.

- **PWA / Mobile Native:** Progressive Web App with offline fallback & Telegram WebApp SDK integration.

Database & Persistence Layer

- **Primary DB:** PostgreSQL 16 (Partitioned tables for bets, ledger transactions, and audit logs).
- **In-Memory Cache:** Redis 7 Cluster (Active order book state, market locks, live exposure tracking).
- **Timeseries Data:** TimescaleDB extension for historical odds and performance telemetry.

Infrastructure & Security

- **Containerization:** Docker + Kubernetes (EKS/ GKE) with auto-scaling rules.
- **Edge Security:** Cloudflare Enterprise (DDoS mitigation, Web Application Firewall, SSL/TLS termination).
- **Data Stream Integration:** Betfair Exchange API / Betfair Historical Data / Betradar WebSocket integrations.

3. TECHNICAL REQUIREMENT DESCRIPTION (TRD) & ENGINEERING SPECS

3.1 Double-Entry Credit Ledger Schema (PostgreSQL)

To ensure 100% mathematical consistency without loss of funds, every financial movement (credit assignment, bet placement, liability lock, settlement) must execute within atomic SQL transactions using strict double-entry ledger rules.

```
-- Core Hierarchy & Ledger Schema
CREATE TYPE role_enum AS ENUM ('ADMIN', 'SUPER_MASTER', 'MASTER', 'AGENT', 'USER');
CREATE TYPE bet_type_enum AS ENUM ('BACK', 'LAY');
CREATE TYPE bet_status_enum AS ENUM ('MATCHED', 'UNMATCHED', 'SETTLED', 'CANCELLED', 'LAPSED');

CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  username VARCHAR(50) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  role role_enum NOT NULL,
  parent_id UUID REFERENCES users(id),
  credit_limit NUMERIC(15, 2) DEFAULT 0.00 CHECK (credit_limit >= 0),
  available_credit NUMERIC(15, 2) DEFAULT 0.00,
  exposure NUMERIC(15, 2) DEFAULT 0.00,
  is_active BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE markets (
  id VARCHAR(100) PRIMARY KEY,
  event_name VARCHAR(255) NOT NULL,
  market_type VARCHAR(50) NOT NULL,
  is_locked BOOLEAN DEFAULT FALSE,
  in_play BOOLEAN DEFAULT FALSE,
```

```

        created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
    );

CREATE TABLE bets (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID REFERENCES users(id) NOT NULL,
    market_id VARCHAR(100) REFERENCES markets(id) NOT NULL,
    selection_id INT NOT NULL,
    type bet_type_enum NOT NULL,
    price NUMERIC(8, 2) NOT NULL,
    stake NUMERIC(15, 2) NOT NULL,
    liability NUMERIC(15, 2) NOT NULL,
    status bet_status_enum DEFAULT 'MATCHED',
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE ledger_entries (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    sender_id UUID REFERENCES users(id),
    receiver_id UUID REFERENCES users(id),
    amount NUMERIC(15, 2) NOT NULL,
    transaction_type VARCHAR(50) NOT NULL, -- 'CREDIT_ALLOCATION', 'BET_LOCK', 'BET_PAYOUT',
    'COMMISSION'
    reference_id VARCHAR(100),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

```

3.2 Real-Time Back/Lay Matching & Exposure Algorithm

When a player places a **LAY** bet at odds P with stake S , the system calculates the liability: $Liability = S * (P - 1)$. The engine must atomically check `available_credit >= Liability`, deduct the liability, and add it to current exposure before confirming the order.

```

// Pseudocode for Atomic Order Validation and Exposure Lock
async function placeLayOrder(userId, marketId, selectionId, price, stake) {
    const liability = stake * (price - 1.0);

    const client = await pool.connect();
    try {
        await client.query('BEGIN ISOLATION LEVEL SERIALIZABLE');

        // Check available credit
        const userRes = await client.query('SELECT available_credit, exposure FROM users WHERE id = $1 FOR UPDATE', [userId]);
        const user = userRes.rows[0];

        if (user.available_credit < liability) {
            throw new Error("INSUFFICIENT_CREDIT");
        }

        // Update balance and exposure
        await client.query(
            'UPDATE users SET available_credit = available_credit - $1, exposure = exposure + $1 WHERE id = $2',
            [liability, userId]
        );
    }
}

```

```
// Insert bet record
const betRes = await client.query(
  'INSERT INTO bets (user_id, market_id, selection_id, type, price, stake, liability,
status) VALUES ($1, $2, $3, $4, $5, $6, $7, $8) RETURNING id',
  [userId, marketId, selectionId, 'LAY', price, stake, liability, 'MATCHED']
);

await client.query('COMMIT');
return { success: true, betId: betRes.rows[0].id };
} catch (err) {
  await client.query('ROLLBACK');
  throw err;
} finally {
  client.release();
}
}
```